

CEUN-IMT - INSTITUTO MAUÁ DE TECNOLOGIA

CURSO DE ENGENHARIA DE COMPUTAÇÃO

ECM516 - ARQUITETURA DE COMPUTADORES

ECM252 - LINGUAGENS DE PROGRAMAÇÃO 2

**SkySwift: Desenvolvimento de uma Infraestrutura de
Entrega Autônoma via Drones baseada em
Microserviços**

Arthur Gama Ruiz - 23.01445-8

Enzo Oliveira D'Onofrio - 23.01561-6

Felipe Fazio da Costa - 23.00055-4

João Vitor Morimoto Sesma - 23.01516-0

Leonardo Souza Olivieri - 23.01512-8

Pedro Wilian Palumbo Bevilacqua - 23.01307-9

SÃO CAETANO DO SUL

2026

Arthur Gama Ruiz, Enzo Oliveira D’Onofrio, Felipe Fazio da
Costa,
João Vitor Morimoto Sesma, Leonardo Souza Olivieri, Pedro
Bevilacqua

SkySwift: Desenvolvimento de uma Infraestrutura de Entrega Autônoma via Drones baseada em Microserviços

Relatório técnico apresentado como requisito parcial para aprovação nas disciplinas de Arquitetura de Computadores (ECM516) e Linguagens de Programação 2 (ECM252) no CEUN-IMT - Instituto Mauá de Tecnologia.

SÃO CAETANO DO SUL
2026

RESUMO

Este trabalho apresenta o projeto SkySwift, uma solução integrada para logística autônoma utilizando drones. A arquitetura proposta rompe com o modelo monolítico tradicional, adotando um ecossistema de microsserviços desacoplados que se comunicam através de um barramento de eventos centralizado. A implementação técnica abrange desde o backend em Node.js com persistência poliglota (MySQL e SQLite), até uma interface de usuário rica desenvolvida em React.js. O foco principal reside na confiabilidade da entrega de mensagens, segurança através de criptografia assimétrica e JWT, e precisão geográfica via algoritmos de roteirização. Os resultados demonstram um sistema resiliente, capaz de processar pedidos, calcular rotas e notificar usuários de forma assíncrona e escalável.

Palavras-chave: Microsserviços, Barramento de Eventos, Node.js, React, Logística de Drones.

Índice

1	INTRODUÇÃO	5
2	ARQUITETURA DE MICROSERVIÇOS	5
2.1	Filosofia de Design	5
2.2	Barramento de Eventos (Event Bus)	5
3	DETALHAMENTO TÉCNICO E CÓDIGO	6
3.1	Barramento de Eventos (Hub Central)	6
3.2	Cálculo de Rota e Rastreamento	6
3.3	Frontend React e Orquestração de Estado	7
3.4	Segurança e Integração Reativa	7
4	INFRAESTRUTURA DE DADOS	8
5	CONCLUSÃO	8

Lista de Figuras

1	Fluxo de Distribuição de Eventos (Pub/Sub)	5
2	Fluxo de Estado e Propagação de Dados no Frontend	7
3	Cadeia de Reação Reativa de Serviços Periféricos	7
4	Modelo de Dados Relacional	8

1 INTRODUÇÃO

O mercado de entregas *last-mile* enfrenta desafios crescentes relacionados à infraestrutura urbana. A proposta do SkySwift é automatizar esse processo utilizando drones, integrando tecnologias de ponta em desenvolvimento de software e arquitetura de sistemas distribuídos. Este relatório detalha a engenharia por trás da plataforma, focando na separação de responsabilidades e na robustez da comunicação entre os módulos.

2 ARQUITETURA DE MICROSERVIÇOS

2.1 Filosofia de Design

Diferente de uma aplicação monolítica, onde uma falha em um módulo pode comprometer todo o sistema, o SkySwift utiliza o isolamento de processos. Cada serviço possui seu próprio contexto, dependências e, em alguns casos, banco de dados.

2.2 Barramento de Eventos (Event Bus)

O coração da arquitetura é o **Barramento de Eventos**. Ele atua como um mediador (*Middleware*) seguindo o padrão *Observer*.

Funcionamento: 1. Um serviço de origem realiza um *POST* para o barramento enviando um objeto JSON que contém o **tipo** do evento e os **dados**. 2. O barramento registra o evento em um histórico. 3. O barramento percorre uma lista interna de serviços "inscritos" e repassa o evento para cada um deles via HTTP.

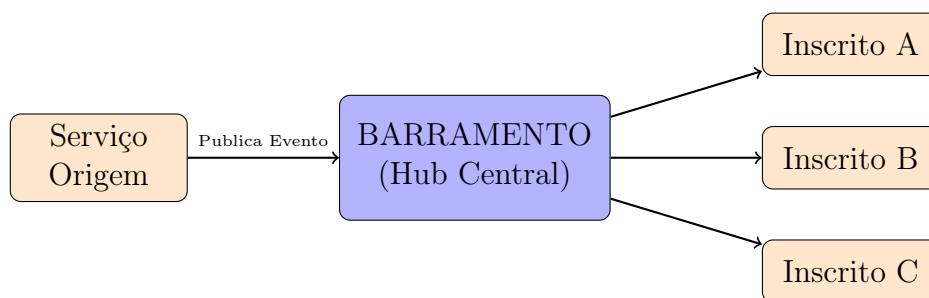


Figura 1: Fluxo de Distribuição de Eventos (Pub/Sub)

3 DETALHAMENTO TÉCNICO E CÓDIGO

3.1 Barramento de Eventos (Hub Central)

O Barramento de Eventos é o sistema nervoso central do ecossistema. Ele gerencia a inscrição de microsserviços e a distribuição de mensagens seguindo o modelo *Publish/Subscribe*. Nesta arquitetura, o serviço emissor (Publisher) não precisa conhecer o endereço ou a existência dos receptores (Subscribers), o que reduz a complexidade de conexão de $O(n^2)$ para $O(n)$.

A implementação do broadcast no arquivo `server.js` do barramento é crítica para a consistência eventual do sistema. Abaixo, o trecho que garante a entrega para todos os serviços ativos:

```
1 async function distribuirEvento(evento) {
2   const resultados = [];
3   for (const servico of inscricoes) {
4     if (servico.nome === evento.origem) continue; // Evita loop infinito
5
6     try {
7       await axios.post(`${servico.url}/eventos/receber`, evento, {
8         timeout: 5000,
9       });
10      resultados.push({ servico: servico.nome, status: 'entregue' });
11    } catch (erro) {
12      resultados.push({ servico: servico.nome, status: 'falha' });
13    }
14  }
15  return resultados;
16 }
```

Listing 1: Implementação de Distribuição de Eventos

3.2 Cálculo de Rota e Rastreamento

O microsserviço `entrega_via_drone` implementa uma estratégia de *fallback* geográfica. Ele tenta consumir o provedor OSRM (*Open Source Routing Machine*), mas possui uma inteligência local baseada na Fórmula de Haversine. Esta abordagem garante que, mesmo sem conectividade com a malha rodoviária externa, o drone consiga estimar o tempo de voo em linha reta.

$$d = 2R \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \quad (1)$$

Onde $a = \sin^2(\Delta\phi/2) + \cos\phi_1 \cdot \cos\phi_2 \cdot \sin^2(\Delta\lambda/2)$.

```
1 function haversineDistanceMeters(lat1, lon1, lat2, lon2) {
```

```

2  const R = 6371000; // Raio da Terra em metros
3  const dLat = (lat2 - lat1) * Math.PI / 180;
4  const dLon = (lon2 - lon1) * Math.PI / 180;
5  const a = Math.sin(dLat/2) * Math.sin(dLat/2) +
6           Math.cos(lat1 * Math.PI / 180) * Math.cos(lat2 * Math.PI /
7           180) *
8           Math.sin(dLon/2) * Math.sin(dLon/2);
9  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
10 return R * c;

```

Listing 2: Implementação da Fórmula de Haversine

3.3 Frontend React e Orquestração de Estado

A interface foi desenvolvida utilizando **React 19** com **Vite**. O grande diferencial técnico reside no roteamento customizado baseado em estado, que evita o uso de bibliotecas externas pesadas e garante transições rápidas entre o painel de pedidos e o mapa de rastreamento.

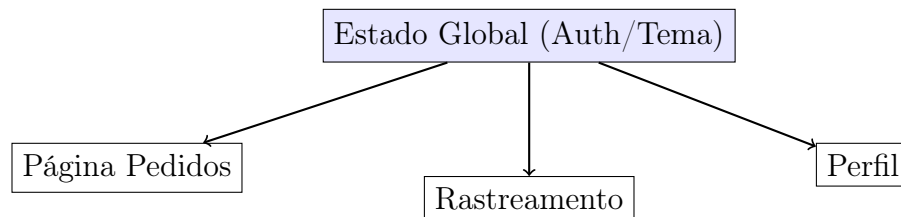


Figura 2: Fluxo de Estado e Propagação de Dados no Frontend

3.4 Segurança e Integração Reativa

O serviço de usuários protege os dados sensíveis utilizando **bcrypt**. A autenticação gera um *JSON Web Token* (JWT) que permite o processamento distribuído (Arquitetura de Computadores), onde cada microsserviço pode validar a identidade do usuário sem sobrecarregar o banco de dados central, utilizando apenas a chave pública de assinatura.

Serviços periféricos como **notificacoes** e **contato_email** funcionam como consumidores reativos puros. Eles processam eventos do tipo **PEDIDO_CRIADO** para disparar e-mails via SMTP e alertas em tempo real.

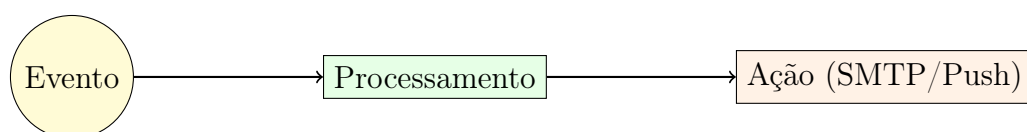


Figura 3: Cadeia de Reação Reativa de Serviços Periféricos

4 INFRAESTRUTURA DE DADOS

O sistema utiliza um esquema relacional para garantir a integridade referencial. Abaixo, o diagrama simplificado das tabelas principais:

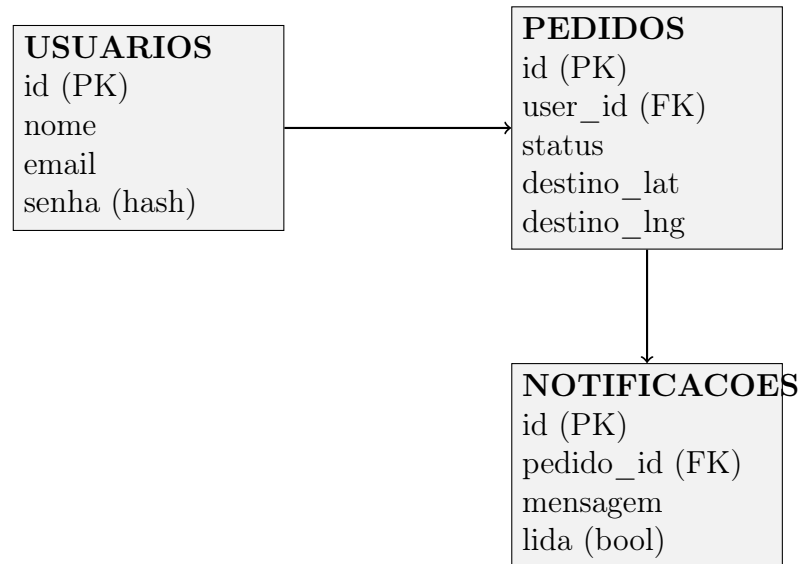


Figura 4: Modelo de Dados Relacional

5 CONCLUSÃO

O SkySwift demonstrou ser uma plataforma robusta para a gestão de entregas autônomas. A escolha pela arquitetura de microsserviços via barramento de eventos provou-se eficaz para o desacoplamento, permitindo que novos serviços (como um gateway de pagamento) sejam adicionados sem alterar o código existente. O projeto atende integralmente aos requisitos de ECM516 e ECM252, consolidando conhecimentos de sistemas distribuídos, segurança e desenvolvimento frontend moderno no CEUN-IMT.